R Data Wrangling & Statistics

A practical quick-reference guide covering essential R concepts — from data structures to statistical testing — illustrated with real code examples.

Download PDF Guide

1 How to Work with Basic Data Structures

R provides several fundamental data structures. Understanding vectors, matrices, and lists is the first step before working with real datasets.

vector matrix list indexing class()

▸ How to create and label a matrix

> Use `rbind()` to stack row vectors into a matrix, then assign row and column names for clarity.

```
# Build two numeric vectors
a ← c(2, 5, 9)
b ← c(17, 25, 33)

# Stack them as rows into a matrix
matr ← rbind(a, b)

# Inspect dimensions and type
dim(matr)    # returns 2 3
class(matr)  # returns "matrix" "array"

# Assign meaningful row and column names
rownames(matr) ← c("Ted", "Robin")
colnames(matr) ← c("Q1", "Q2", "Q3")
matr
```

Made with Manus

How to check the type of an object with class()

Use `class()` to inspect what kind of object you are working with. This is especially useful after reading data or performing transformations, when the type may not be obvious.

```
# Check the type of different objects
class(matr)      # "matrix" "array"
class(my_list)   # "list"
class(words)     # "character"
class(42L)       # "integer"
class(3.14)      # "numeric"

# Useful when reading external data — check if a column is numeric or labelle
class(wvs6$V2)   # "haven_labelled" "vctrs_vctr" "double"
```

How to create a list and index its elements

A list can hold objects of different types. Use double brackets [[i]] to extract an element, then standard indexing to drill further inside.

```
# Create a character vector
words ← c("Somebody", "Once", "Told", "Me", "The", "World", "Is")

# Combine objects of different types into a list
my_list ← list(matr, words, 42)

# Extract the 1st element (the matrix)
my_list[[1]]

# Extract the 2nd element (the vector), then its 1st item
my_list[[2]][1]       # "Somebody"

# Extract cell (1,2) from the matrix stored in the list
my_list[[1]][1, 2]    # row 1, column 2

# Extract the last element of the vector in the list
my_list[[2]][length(my_list[[2]])]  # "Is"
```

Tip — single vs double brackets

Made with Manus

`my_list[1]` returns a list of length 1; `my_list[[1]]` returns the actual object inside. Always use `[[]]` when you need to work with the content directly.

2 How to Create and Sort a Data Frame

Data frames are the workhorse of R data analysis. They combine vectors of equal length into a table where each column can have its own type.

data.frame NA order() head() rownames() cbind()

How to build a data frame from vectors

Pass named vectors to data.frame(). Missing values are represented as NA and are handled gracefully by most R functions.

```
country   ← c("Japan", "Mexico", "South Korea", "China", "Latvia",
               "Lithuania", "Austria", "Russia", "Canada", "Portugal")
Work      ← c(25.2, 24.2, 23.9, 23.6, 22.9, 21.9, 21.3, 21.2, 21.2, 20.5)
Housework ← c(9.2, 18.3, 9.1, 11.4, 13.6, 15.6, NA, 16.7, 12.9, 15.5)

# Combine into a data frame
timeuse ← data.frame(country, Work, Housework)
timeuse
```

How to add a computed column

Use the $ operator to create a new column as a function of existing ones. NA values propagate automatically — any row with an NA input will produce an NA output.

```
# Add total labor (paid + unpaid) as a new column
timeuse$labor ← timeuse$Work + timeuse$Housework
timeuse
```

How to assign row names and combine data frames side by side

Made with Manus

Use `rownames()` to give meaningful labels to the rows of a data frame — this is especially useful when each row represents a scenario (e.g. a sample size). Use `cbind()` to join two or more data frames horizontally, provided they have the same number of rows.

```
# Assign row labels to a CI results data frame
ci_95_3000 ← data.frame(Lower = ci.l, Mean = 5500, Upper = ci.u)
rownames(ci_95_3000) ← c("ss100", "ss500", "ss1000", "ss10000", "ss20000", '
ci_95_3000

# Repeat for a second scenario (sd = 100)
ci_95_100 ← data.frame(Lower = ci.l2, Mean = 5500, Upper = ci.u2)
rownames(ci_95_100) ← c("ss100", "ss500", "ss1000", "ss10000", "ss20000", "s

# Combine both result tables side by side into one summary table
all_ci ← cbind(ci_95_3000, ci_95_100)
all_ci
```

Tip — cbind() vs merge()

Use `cbind()` when the rows of the data frames already correspond to each other in order. Use `merge()` when you need to join on a shared key column, similar to a SQL JOIN.

▸ How to sort a data frame by a column

order() returns the row indices that would sort the data frame. Pass it inside square brackets to reorder the rows. Use decreasing = TRUE for descending order, then head() to see the top rows.

```
# Sort by total labor, descending
timeuse_sorted ← timeuse[order(timeuse$labor, decreasing = TRUE), ]
timeuse_sorted

# Show only the top 3 rows
head(timeuse_sorted, 3)
```

Tip — NA handling in order()

By default, `order()` places `NA` values at the end. Use `na.last = FALSE` to
the top, or `na.last = NA` to remove them from the result.

Made with Manus

3 How to Read External Data and Work with Labelled Variables

Survey data is often distributed in SPSS .sav format. The haven package reads these files and preserves variable labels and value labels as metadata.

haven read_sav() as_factor() attributes() subsetting

How to load a labelled SPSS file

haven::read_sav() imports the file and keeps value labels attached to each variable. Use dim() to confirm the number of rows and columns loaded.

```
library("haven")

# Read an SPSS .sav file — labels are preserved as metadata
wvs6 ← read_sav("WV6_Data_sav_v20201117.sav")
dim(wvs6)   # e.g. 89565 respondents, 440 variables
```

How to inspect variable labels and values

attributes() reveals the full metadata of a labelled variable. print_labels() from haven gives a cleaner view of value-label pairs.

```
# Inspect metadata of a variable
attributes(wvs6$V2)   # shows label, labels (value-label pairs), format, etc.

# Print value labels in a readable format
print_labels(wvs6$V2)

# Convert a labelled integer column to a factor with labels as levels
head(as_factor(wvs6$V2))
```

How to subset rows and columns

Made with Manus

> Filter rows using logical conditions inside square brackets. Select specific columns by passing a character vector of column names. Both operations can be combined in a single step.

```
# Keep only respondents from Russia (code 643) or Sweden (code 752)
wvs6_rusw <- wvs6[wvs6$V2 == 643 | wvs6$V2 == 752, ]

# Equivalently, using labels instead of numeric codes
wvs6_rusw <- wvs6[as_factor(wvs6$V2) == "Russia" |
                  as_factor(wvs6$V2) == "Sweden", ]

# Keep only the variables we need
wvs6_rusw <- wvs6_rusw[, c("V2", "V105", "V248", "V59", "V237")]
dim(wvs6_rusw)   # 3706 respondents, 5 variables
```

4 How to Compute Frequency Tables and Descriptive Statistics

The choice of descriptive statistic depends on the measurement scale. Nominal and ordinal variables call for frequency tables and percentages; metric variables call for means, medians, and standard deviations.

table() prop.table() addmargins() summary() useNA complete.cases()

▸ How to build a frequency table with missing values

> Always include useNA = 'ifany' so that missing values appear explicitly in the table rather than being silently dropped.

```
# Absolute frequencies — NAs shown explicitly
table(as_factor(wvs6$V2), useNA = "ifany")

# Add row/column totals with addmargins()
addmargins(table(as_factor(wvs6$V2), useNA = "ifany"))
```

Made with Manus

▸ How to compute percentage distributions

Wrap `table()` inside `prop.table()` to get proportions, then multiply by 100 and round for a clean percentage output. The optional **margin** argument controls whether proportions sum to 1 across the whole table (no argument), within each row (`margin = 1`), or within each column (`margin = 2`).

```
# Overall proportions (sums to 1 across the whole table)
round(prop.table(table(as_factor(wvs6$V2), useNA = "ifany")) * 100, 2)

# Column proportions (margin = 2): share of each happiness level WITHIN each
hap_share ← round(
  prop.table(
    table(wvs6_rusw$happy_rec,
          droplevels(as_factor(wvs6_rusw$V2))),
    margin = 2   # proportions sum to 1 within each country column
  ), 3)
hap_share
# hap_share[1, 1] = share of "Not happy" in Russia
# hap_share[1, 2] = share of "Not happy" in Sweden

# Convert to a data frame for easier viewing or export
as.data.frame(
  round(prop.table(table(as_factor(wvs6$V2), useNA = "ifany")) * 100, 2)
)
```

Tip — choosing the margin

Use `margin = 2` (column proportions) when you want to compare the distribution of a variable *within* each group — for example, the share of happy/unhappy people separately for Russia and Sweden. Use `margin = 1` (row proportions) when you want to see how each category is distributed across groups.

▸ How to count valid (non-missing) observations with complete.cases()

`complete.cases()` returns a logical vector that is `TRUE` for every row with no missing values. Wrapping it in `table()` gives a quick count of valid vs missing cases — essential before computing confidence intervals or running tests that require a valid sample size *n*.

```
# Count valid vs missing cases overall
table(complete.cases(wvs6_rusw$trust_gen_cnt))
# FALSE  TRUE
#    12  3694
```

Made with Manus

```
# Count valid cases per country group (used as n in CI formulas)
gtval ← table(
  complete.cases(wvs6_rusw$trust_gen_cnt),
  droplevels(as_factor(wvs6_rusw$V2))
)
gtval
#        Russia Sweden
# FALSE       8      4
# TRUE     1498   2196

gtval[2, 1]   # valid n for Russia: 1498
gtval[2, 2]   # valid n for Sweden: 2196

# Same approach for a binary variable (e.g. happiness)
hap_val ← addmargins(
  table(complete.cases(wvs6_rusw$happy_rec),
        droplevels(as_factor(wvs6_rusw$V2))), 1)
hap_val[2, 1]   # valid n for Russia
```

Why this matters for CIs

When computing a confidence interval, you need the number of *valid* responses, not the total number of rows. `complete.cases()` combined with `table()` gives you the exact valid *n* per group, which you then use in the CI formula as the denominator.

▸ How to summarise a metric variable

> summary() returns the five-number summary plus the mean in a single call. It is the go-to function for a quick overview of any continuous variable.

```
# Five-number summary + mean for a metric variable
summary(wvs5_rus$V237)   # Min, Q1, Median, Mean, Q3, Max, NAs

# Group-wise summaries using tapply
tapply(wvs5_rus$god, as_factor(wvs5_rus$sex), summary)
```

Made with Manus

5 How to Recode Variables

Recoding is one of the most common data preparation tasks. It covers reversing scale direction, collapsing categories, creating dummy variables, and assigning new labels. Collapsing many fine-grained categories into a smaller number of broader groups is not just a simplification choice — it is also a practical necessity when some categories contain very few observations. Statistical tests become unreliable with small cell sizes, so merging sparse categories into larger buckets produces more stable and trustworthy results.

car::recode() scales::rescale() labelled() dummy variable scale reversal cor()

▸ How to reverse a Likert scale

> For a scale running from 1 to k, the reversed value is (k + 1) − original. Always verify the reversal with a cross-tabulation before proceeding.

```
# V105 is a 4-point scale (1=Trust completely ... 4=Do not trust at all)
# We want 1=Do not trust at all, 4=Trust completely
wvs6_rusw$trstr ← 5 - as.numeric(wvs6_rusw$V105)

# Assign meaningful labels to the reversed variable
library("haven")
wvs6_rusw$trstr ← labelled(wvs6_rusw$trstr,
  c("Do not trust at all"    = 1,
    "Do not trust very much" = 2,
    "Trust somewhat"         = 3,
    "Trust completely"       = 4))

# Verify: original and recoded values should mirror each other
table(as_factor(wvs6_rusw$V105), as_factor(wvs6_rusw$trstr))
```

▸ How to collapse categories with car::recode()

> car::recode() accepts a string of rules separated by semicolons. Ranges are written as lo:hi. This is ideal for merging scale points or creating ordinal groupings.

```
library(car)

# Collapse a 10-point financial satisfaction scale into 5 grou
wvs6_rusw$fin_sat_rec ← car::recode(wvs6_rusw$V59,
```

Made with Manus

```
  "1:2=1; 3:4=2; 5:6=3; 7:8=4; 9:10=5")

# Assign labels to the new variable
wvs6_rusw$fin_sat_rec <- labelled(wvs6_rusw$fin_sat_rec,
  c("Dissatisfied"       = 1,
    "Rather Dissatisfied"= 2,
    "Neither"            = 3,
    "Rather Satisfied"   = 4,
    "Satisfied"          = 5))

# Check the recoding
table(droplevels(as_factor(wvs6_rusw$V59)),
      as_factor(wvs6_rusw$fin_sat_rec), useNA = "ifany")
```

How to create a dummy (binary) variable

> Dummy variables encode a categorical distinction as 0/1. They are required for many statistical models and for clear cross-tabulation comparisons.

```
# Recode education into: 0 = no university, 1 = university or higher
wvs6_rusw$educ_rec <- car::recode(wvs6_rusw$V248, "1:7=0; 8:9=1")

wvs6_rusw$educ_rec <- labelled(wvs6_rusw$educ_rec,
  c("No university educ" = 0,
    "University educ"    = 1))

# Verify
table(droplevels(as_factor(wvs6_rusw$V248)),
      as_factor(wvs6_rusw$educ_rec), useNA = "ifany")
```

How to rescale a variable to [0, 1]

> scales::rescale() linearly maps any numeric range to a target interval. This is useful when building composite indices from variables measured on different scales.

```
library(scales)

# Rescale church attendance (reversed) to [0, 1]
# to = c(1, 0) because the original scale is reversed
wvs5_rus$att <- round(scales::rescale(as.numeric(wvs5_rus$V186,
```

Made with Manus

```
                                        to = c(1, 0)), 4)

# Verify: check original vs rescaled values
table(as_factor(wvs5_rus$V186), wvs5_rus$att, useNA = "ifany")
```

How to verify a rescaling with cor()

> After rescaling a variable, use `cor()` to confirm that the transformation preserved the original ordering. If the rescaling is correct, the Pearson correlation between the original and the transformed variable should be exactly **1** (or **−1** if the scale was reversed). A value different from ±1 would indicate an error in the recoding.

```
library(scales)

# Rescale age to [0, 1] — same direction, no reversal
wvs5_rus$age ← round(scales::rescale(as.numeric(wvs5_rus$V237), to = c(0, 1)

# Verify: correlation between original and rescaled should be exactly +1
cor(wvs5_rus$V237, wvs5_rus$age)
# [1] 1   → perfect positive correlation confirms correct rescaling

# For a reversed scale (to = c(1, 0)), correlation should be -1
wvs5_rus$att ← round(scales::rescale(as.numeric(wvs5_rus$V186), to = c(1, 0)
cor(as.numeric(wvs5_rus$V186), wvs5_rus$att)
# [1] -1  → perfect negative correlation confirms correct reversal
```

When to expect −1 vs +1

If you rescaled with `to = c(0, 1)` (same direction as original), `cor()` should return **+1**. If you reversed the scale with `to = c(1, 0)`, it should return **−1**. Either result confirms the rescaling is a perfect linear transformation of the original — no information was lost or distorted.

6 How to Build a Composite Index

A composite index summarises several related indicators into a single score. common approach is to average the component variables row-wise after ensuring they all

Made with Manus

point in the same direction.

rowMeans() is.nan() index construction plot() rcorr() psych::alpha()

Cronbach's alpha

How to compute a row-wise mean index

rowMeans() computes the mean across selected columns for each row. Setting na.rm = TRUE allows partial responses to contribute to the index. Replace any resulting NaN (all components missing) with NA.

```
# Index of generalised trust: average of three trust items
# (all rescaled to [1,4] and pointing in the same direction)
wvs6_rusw$trust_gen_cnt ← round(
  rowMeans(wvs6_rusw[c("trunknown", "trnat", "trrel")], na.rm = TRUE), 2)

# Replace NaN (produced when ALL three items are NA) with NA
wvs6_rusw$trust_gen_cnt[is.nan(wvs6_rusw$trust_gen_cnt)] ← NA

# Quick summary of the new index
summary(wvs6_rusw$trust_gen_cnt)

# Scatter plots to visually verify the index correlates with its components
plot(wvs6_rusw$trust_gen_cnt, wvs6_rusw$trunknown)
plot(wvs6_rusw$trust_gen_cnt, wvs6_rusw$trnat)
plot(wvs6_rusw$trust_gen_cnt, wvs6_rusw$trrel)
```

Tip — NaN vs NA

`rowMeans( ... , na.rm = TRUE)` returns `NaN` when every value in a row is `NA`. Use `is.nan()` to detect and replace these with `NA` so that downstream functions treat them correctly as missing.

How to create a quick scatter plot with base R plot()

Base R's `plot(x, y)` produces a scatter plot instantly with no extra packages. It is the fastest way to visually verify a relationship — for example, checking that each component of an index correlates with the composite score before publishing results.

```
# Basic scatter plot: x = composite index, y = one of its components
plot(wvs6_rusw$trust_gen_cnt, wvs6_rusw$trunknown)
# If the index is correct, points should follow a clear diagonal pattern
```

Made with Manus

```
# Add axis labels and a title for clarity
plot(wvs6_rusw$trust_gen_cnt, wvs6_rusw$trunknown,
     xlab = "Generalised Trust Index",
     ylab = "Trust in Unknown People",
     main = "Index vs. Component Verification")

# Repeat for each component to confirm all correlate as expected
plot(wvs6_rusw$trust_gen_cnt, wvs6_rusw$trnat)
plot(wvs6_rusw$trust_gen_cnt, wvs6_rusw$trrel)
```

plot() vs ggplot2

`plot()` is ideal for quick exploratory checks during data preparation. Use `ggplot2` when you need publication-quality graphics with precise control over colours, labels, and themes.

▸ How to compute a correlation matrix with significance tests using rcorr()

After building a composite index, use `Hmisc::rcorr()` to check that all component variables correlate meaningfully with each other and with the index. It returns three matrices: **r** (Pearson correlations), **n** (pairwise valid sample sizes), and **P** (p-values). All components should show positive, significant correlations with the index — if one does not, consider removing it.

```
library(Hmisc)

# Correlation matrix for the index and all its components
Hmisc::rcorr(as.matrix(wvs5_bul[, c("ins", "pol", "cou", "gov", "par", "parl'
# Output includes three matrices:
# $r  — Pearson correlation coefficients
# $n  — pairwise valid sample sizes
# $P  — p-values (NA on diagonal)

# Interpretation:
# All component-to-index correlations should be positive and significant (P <
# High inter-item correlations (r > 0.4) suggest the items measure the same c
# A component with a low or non-significant correlation is a candidate for re
```

Tip — as.matrix() is required

`rcorr()` requires a numeric matrix as input. Always wrap the data frame s
`as.matrix()` before passing it. Columns with all-NA values will produce `NA`

Made with Manus

output.

How to assess index reliability with Cronbach's alpha

Use `psych::alpha()` to measure the **internal consistency** of a composite index — i.e., whether all items reliably measure the same underlying construct. The key output is **std.alpha**: values above 0.70 are generally considered acceptable, above 0.80 good, and above 0.90 excellent. The table also shows what std.alpha would become if each item were dropped, helping you decide whether to remove a weak item.

```
library(psych)

# Compute Cronbach's alpha for the five institutional confidence items
psych::alpha(wvs5_bul[, c("pol", "cou", "gov", "par", "parl")])

# Key output to read:
# std.alpha = 0.88  → good internal consistency
#
# 'Reliability if an item is dropped' table:
# Each row shows std.alpha if that item were removed
# If all values are LOWER than 0.88, no item should be dropped
# If a row shows a HIGHER value, that item is weakening the scale

# Example interpretation:
# std.alpha = 0.88, and removing any single item makes it worse
# → All five items contribute positively; keep the full index
```

Rule of thumb for item removal

If dropping an item would *increase* std.alpha noticeably, that item is weakening the scale and should be considered for removal. If dropping any item would leave std.alpha the same or worse, the current set of items is optimal — keep them all.

7 How to Visualise Distributions with ggplot2

The ggplot2 package builds plots layer by layer. A bar chart is the standard c
categorical and ordinal variables; the fill aesthetic enables easy group compa.

Made with Manus

ggplot2 geom_bar() labs() scale_fill_manual() ggarrange()

How to create a grouped bar chart

> Use position = 'dodge' to place bars side by side for group comparison. scale_fill_manual() lets you specify exact hex colours for each group.

```
library(ggplot2)

finsat ← ggplot(wvs6_rusw,
                 aes(x = as_factor(fin_sat_rec), fill = as_factor(V2))) +
  geom_bar(position = "dodge") +
  labs(
    title    = "Financial satisfaction in Sweden and Russia",
    subtitle = "Sample: Sweden and Russia",
    caption  = "Data: WVS, 6th wave",
    x        = "Satisfaction with Finance",
    y        = "Frequency"
  ) +
  scale_fill_manual(
    name   = "Country",
    values = c("Russia" = "#1a4784", "Sweden" = "#C20061")
  ) +
  theme_minimal()

print(finsat)
```

How to create a proportional stacked bar chart

> position = 'fill' normalises each bar to 100%, making it easy to compare the relative distribution of a variable across groups regardless of group size.

```
ggplot(wvs6_swru, aes(x = fam, fill = as_factor(V2))) +
  geom_bar(position = "fill") +
  scale_fill_brewer(palette = "Pastel1") +
  labs(
    title    = "Important in life: Family",
    subtitle = "Pooled sample: Russia and Sweden",
    caption  = "Data: WVS, 6th wave",
    x        = "How important is family in your life?",
    y        = "Share"
```

Made with Manus

```
)
```

▸ How to combine multiple plots into one figure

> ggpubr::ggarrange() arranges several ggplot objects into a single panel figure, which is ideal for comparing distributions across variables in one view.

```
library(ggpubr)

# Arrange four plots in a 2×2 grid
ggarrange(finsat, savings, edu, trust)
```

8 How to Test for Normality

Many parametric statistical tests assume that the data follow a normal distribution. Before choosing a test, it is good practice to check this assumption both visually and formally.

hist() ggqqplot() shapiro.test() normality group subsetting

▸ How to inspect the distribution within a specific group

> Use square-bracket subsetting inside `hist()` to plot the distribution for one group at a time. This is the quickest way to visually check whether normality holds separately for each category before running a formal test or choosing a statistical test.

```
# Plot the distribution of institutional confidence for each religiosity grou
hist(wvs5_rus$ins[wvs5_rus$relg == "A convinced atheist"],
     main = "Institutional confidence — Atheists",
     xlab = "Index value")
# Looks not normal

hist(wvs5_rus$ins[wvs5_rus$relg == "A religious person"],
     main = "Institutional confidence — Religious",
     xlab = "Index value")
# Looks not normal
```

Made with Manus

```
hist(wvs5_rus$ins[wvs5_rus$relg == "Not a religious person"],
     main = "Institutional confidence — Non-religious",
     xlab = "Index value")
# Looks not normal

# Same approach for a numeric grouping variable (e.g. sex coded 0/1)
hist(wvs5_rus$god[wvs5_rus$sex == 0],
     main = "Importance of God — Males", xlab = "Rescaled score")
hist(wvs5_rus$god[wvs5_rus$sex == 1],
     main = "Importance of God — Females", xlab = "Rescaled score")
```

Tip — label your plots

Always pass `main=` and `xlab=` so you can tell the group-specific plots apart, especially when generating several in a row.

▸ How to visually inspect normality

A histogram gives a quick impression of the shape. A Q-Q plot compares the empirical quantiles of the data against the theoretical quantiles of a normal distribution — points should fall on the diagonal line if normality holds.

```
library(ggpubr)

# Histogram
hist(wvs5_rus$ins,
     main = "Distribution of Institutional Confidence",
     xlab = "Index value")

# Q-Q plot (ggpubr wrapper around qqnorm/qqline)
ggqqplot(data = wvs5_rus, x = "ins")

# Faceted Q-Q plot by group
ggqqplot(data = wvs5_rus[complete.cases(wvs5_rus$ins), ],
         x = "ins", facet.by = "relg")
```

▸ How to run the Shapiro-Wilk normality test

The null hypothesis of shapiro.test() is that the data come from a normal distribution. A p-value below your significance threshold (e.g., 0.05) leads you to reject r
tapply() to run the test for each group simultaneously.

Made with Manus

```
options(scipen = 999, digits = 8)   # suppress scientific notation

# Overall test
shapiro.test(wvs5_rus$ins)
# W = 0.96901, p-value < 0.001 → reject normality

# Group-wise test (one result per religiosity group)
tapply(wvs5_rus$ins, wvs5_rus$relg, shapiro.test)
# If all p-values < 0.05, normality is rejected in every group
```

Note on sample size

The Shapiro-Wilk test is sensitive to large samples — even trivial deviations from normality will be flagged as significant when n is large. Always combine the formal test with a visual inspection.

9 How to Run Bivariate Tests

First identify the type of variables A and B in your data sample to find the right test in the Statistical Selection Table below. If you have doubts, refer to the examples in the Use Case Examples table. Then check the test conditions (e.g. data normality) using the Conditions & Diagnostics table. Finally, review the Post-Hoc Tests table in case extra steps are required after running the test.

kruskal.test() wilcox.test() pairwise.wilcox.test() chisq.test() aov() TukeyHSD() t.test() cor.test() p-value Holm correction

Statistical Selection Table

If Variable A is...	And Variable B is...	Data Condition	Use this Test	R Function
Nominal (1 Group vs. Value)	Numerical	Normal	One-sample t-test	`t.test(B, mu = value)`

Made with Manus

If Variable A is...	And Variable B is...	Data Condition	Use this Test	R Function
Nominal (1 Group vs. Value)	Numerical / Ordinal	Non-Normal	Sign Test	`BSDA::SIGN.test(B)`
Nominal (2 Indep. Groups)	Numerical	Normal + Equal Var.	Student's t-test	`t.test(B ~ A, var.equal=TRUE)`
Nominal (2 Indep. Groups)	Numerical	Normal + Unequal Var.	Welch's t-test	`t.test(B ~ A, var.equal=FALSE)`
Nominal (2 Indep. Groups)	Numerical	Non-Normal	Mann-Whitney U (Rank Sum)	`wilcox.test(B ~ A)`
Nominal (2 Indep. Groups)	Ordinal	Ordered Scale	Mann-Whitney U (Rank Sum)	`wilcox.test(B ~ A)`
Nominal (2 Related/ Paired)	Numerical	Normal	Paired t-test	`t.test(B1, B2, paired=T)`
Nominal (2 Related/ Paired)	Numerical	Non-Normal	Wilcoxon Signed Rank	`wilcox.test(B1, B2, paired=T)`
Nominal (2 Related/ Paired)	Ordinal	Ordered Scale	Wilcoxon Signed Rank	`wilcox.test(B1, B2, paired=T)`
Nominal (3+ Indep. Groups)	Numerical	Normal + Equal Var.	One-Way ANOVA	`summary(aov(B ~ A))`
Nominal (3+ Indep. Groups)	Numerical	Normal + Unequal Var.	Welch's ANOVA	`oneway.test(B ~ A, var.equal=FALSE)`
Nominal (3+ Indep.	Numerical	Non-Normal	Kruskal-Wallis	`kruskal`

Made with Manus

If Variable A is...	And Variable B is...	Data Condition	Use this Test	R Function
Groups)				
Nominal (3+ Indep. Groups)	Ordinal	Ordered Scale	Kruskal-Wallis	`kruskal.test(B ~ A)`
Nominal	Nominal	Categories	Chi-Square (χ^2)	`chisq.test(table(A, B))`
Numerical	Numerical	Normal / Linear	Pearson Correlation	`cor.test(A, B, method="pearson")`
Numerical	Numerical	Non-Normal	Spearman Correlation	`cor.test(A, B, method="spearman")`
Ordinal	Ordinal	Ordered Scale	Spearman Correlation	`cor.test(A, B, method="spearman")`

Use Case Examples

If Variable A (The Groups) is...	And Variable B (The Measurement) is...	Real-World "For Example"	The Statistical Goal
Nominal (1 Group)	Numerical	*Comparing Exam Scores (B) to a Target Score of 75 (A).*	Compare to a "Value"
Nominal (2 Indep.)	Numerical	*Comparing Wages (B) across Gender: Men vs. Women (A).*	Compare 2 Means
Nominal (2 Indep.)	Ordinal	*Comparing Star Ratings 1–5 (B) across iPhone vs. Android users (A).*	Compare 2 Ranks
Nominal (2 Paired)	Numerical	*Measuring Blood Pressure (B) of the same patients Before vs. After a pill (A).*	Compare Changes

Made with Manus

If Variable A (The Groups) is...	And Variable B (The Measurement) is...	Real-World "For Example"	The Statistical Goal
Nominal (3+ Indep.)	Numerical	*Comparing Exam Scores (B) across Biology, Math, and History majors (A).*	**Compare 3+ Means**
Nominal (3+ Indep.)	Ordinal	*Comparing Happiness Level: Low/Med/High (B) across 4 different Cities (A).*	**Compare 3+ Ranks**
Nominal	Nominal	*Does Smoking: Yes/No (A) relate to Lung Cancer: Yes/No (B)?*	**Look for Association**
Numerical	Numerical	*Does Study Hours (A) relate to Exam Score (B)?*	**Look for a Trend**
Ordinal	Ordinal	*Does Education Level 1–4 (A) relate to Job Seniority 1–5 (B)?*	**Look for a Rank Trend**

Test Conditions & Diagnostics

Test Name	What is it for?	The "Goal" for the Student	R Function
Shapiro-Wilk	Checking Normality.	If $p < 0.05$, the data is Non-Normal → use the non-parametric row in the selection table.	`shapiro.test()`
Levene's Test	Checking Equality of Variance.	Do the groups have similar "spread"? If $p < 0.05$, variances are not equal.	`leveneTest()`
Bartlett's Test	Checking Equality of Variance.	Similar to Levene's, but very sensitive to non-normal data — only use when normality holds.	`bartlett.test()`

Made with Manus

Test Name	What is it for?	The "Goal" for the Student	R Function
Fligner-Killeen	Checking Equality of Variance.	Use this if you already know your data is Non-Normal — robust alternative to Bartlett's.	`fligner.test()`

Post-Hoc Tests (Finding the "Winner")

ANOVA or Kruskal-Wallis tells you "at least one group is different." These follow-up tests identify *which specific pairs* differ (e.g., is it Biology vs. Math, or Math vs. History?).

Test Name	What is it for?	When to use it?	R Function
Tukey HSD	Comparing all pairs.	After a Normal ANOVA.	`TukeyHSD()`
Games-Howell	Comparing all pairs.	After ANOVA when variances are NOT equal.	`games_howell_test()`
Dunn's Test	Comparing all pairs.	After a Kruskal-Wallis test (non-parametric).	`kwAllPairsDunnTest()`
Dunnett's Test	Comparing to a control.	When you only care about comparing "Treatment" vs "Placebo".	`DunnettTest()`

▸ **How to run a Kruskal-Wallis test (3+ groups)**

> Use when the outcome is not normally distributed and you have three or more independent groups. A significant result means at least one group differs — a post-hoc test is then needed to identify which pairs differ.

Made with Manus

```
# Is institutional confidence associated with religiosity (3 groups)?
kruskal.test(wvs5_rus$ins ~ wvs5_rus$relg)
# chi-squared = 19.39, df = 2, p-value = 0.000061
# → At least one religiosity group has a significantly different
#   level of institutional confidence (p < 0.001)

# Post-hoc pairwise comparisons with Holm correction for multiple testing
pairwise.wilcox.test(wvs5_rus$ins, wvs5_rus$relg, p.adjust.method = "holm")
# → The religious vs atheist pair is significant at the 1% level (p = 0.0004
```

▸ How to run a Mann-Whitney U test (2 groups)

> The non-parametric alternative to the independent-samples t-test. Use it when the outcome is not normally distributed in at least one of the two groups.

```
# Is importance of God associated with gender?
# First, check distributions per group
tapply(wvs5_rus$god, as_factor(wvs5_rus$sex), summary)
# Median for males: 0.44 | Median for females: 0.67

# Run the Wilcoxon rank-sum test (= Mann-Whitney U)
wilcox.test(wvs5_rus$god ~ wvs5_rus$sex)
# W = 297776, p-value < 0.001
# → Russian females have a significantly higher importance of God
#   than Russian males (p < 0.001)
```

Interpreting p-values

A p-value below 0.05 means the observed difference would occur by chance less than 5% of the time if the null hypothesis were true. A p-value below 0.01 provides stronger evidence (99% confidence level). Always report both the test statistic and the p-value, and describe the direction of the effect.

▸ How to run a Chi-square test of independence

> Use `chisq.test()` to test whether two **categorical** variables are statistically independent. The null hypothesis is that the two variables are not associated. A significant result (p < 0.05) means the distribution of one variable differs across the categories of the other. Always check that expected cell counts are at least 5 — if not, use Fisher's exact test instead.

Made with Manus

```
# Is religious affiliation associated with gender?
# Step 1: build a contingency table
tbl ← table(as_factor(wvs5_rus$V235),   # rows: sex
            as_factor(wvs5_rus$V187))   # cols: religious person
addmargins(tbl)

# Step 2: check expected cell counts (all should be ≥ 5)
chisq.test(tbl)$expected

# Step 3: run the test
chisq.test(tbl)
# X-squared = 12.34, df = 2, p-value = 0.0021
# → p < 0.05: the distribution of religious affiliation
#   differs significantly between males and females

# Step 4: inspect column proportions to describe the direction
round(prop.table(tbl, margin = 2) * 100, 1)
# Females are more likely to identify as religious (55%) than males (43%)
```

Expected counts rule

Run `chisq.test(tbl)$expected` to inspect expected cell frequencies. If any cell is below 5, the chi-square approximation becomes unreliable. In that case, collapse small categories first (see Section 5) or switch to `fisher.test(tbl)`.

How to run a one-way ANOVA (3+ groups, normal distribution)

> Use `aov()` when the outcome is **continuous and normally distributed** and you want to compare means across three or more independent groups. ANOVA tests whether at least one group mean differs from the others. If the overall test is significant, use `TukeyHSD()` for post-hoc pairwise comparisons with automatic correction for multiple testing.

```
# Is mean age different across education levels?
# Step 1: check normality per group (see Section 8)
tapply(wvs5_rus$V237, as_factor(wvs5_rus$V238), shapiro.test)
# Assume all p-values > 0.05 — normality holds in each group

# Step 2: check homogeneity of variances
library(car)
leveneTest(wvs5_rus$V237 ~ as_factor(wvs5_rus$V238))
# p > 0.05 → variances are not significantly different, ANOVA
```

Made with Manus

```
# Step 3: run one-way ANOVA
fit ← aov(wvs5_rus$V237 ~ as_factor(wvs5_rus$V238))
summary(fit)
# Df  Sum Sq  Mean Sq  F value  Pr(>F)
#  7  123456   17637    45.2    <0.001
# → F is large and p < 0.001: at least one education group
#   has a significantly different mean age

# Step 4: post-hoc pairwise comparisons (Tukey HSD)
TukeyHSD(fit)
# Each row shows the mean difference between a pair of groups,
# its 95% CI, and the adjusted p-value
# → Pairs with p adj < 0.05 differ significantly
```

ANOVA vs Kruskal-Wallis

ANOVA assumes normality within each group and homogeneity of variances. If either assumption fails, use `kruskal.test()` instead. You can check variance homogeneity with `bartlett.test()` or `leveneTest()` from the **car** package.

▸ How to test for equality of variances with leveneTest()

> Run Levene's test **before** an independent t-test or ANOVA to decide whether to use the equal-variance or unequal-variance variant. The null hypothesis is that all groups have the same variance. A p-value below 0.05 means variances are **not** equal — use Welch's t-test or Welch's ANOVA instead.

```
library(car)

# Test equality of variances across marital status groups
leveneTest(wvs6_rus$age ~ wvs6_rus$mar)
# Levene's Test for Homogeneity of Variance (center = median)
#       Df  F value  Pr(>F)
# group  5   8.432   <0.001 ***
# → p < 0.05: variances are NOT equal across groups
# → use Welch's ANOVA (oneway.test with var.equal=FALSE)

# For exactly 2 groups (e.g., gender)
leveneTest(wvs6_rus$age ~ droplevels(as_factor(wvs6_rus$V240)), center = mear
# p > 0.05 → variances are equal → use Student's t-test
```

Made with Manus

Default centre

By default leveneTest() uses the group median as the centre, which is more robust to outliers. Pass center = mean to replicate the classic Levene's test.

How to test for equality of variances with fligner.test()

Fligner-Killeen is a non-parametric alternative to Levene's test. Use it when you already know the data is non-normal, since Bartlett's test is sensitive to departures from normality. The null hypothesis is the same: equal variances across all groups.

```
# Fligner-Killeen test (robust to non-normality)
fligner.test(wvs6_rus$age ~ wvs6_rus$mar)
# Fligner-Killeen test of homogeneity of variances
# Fligner-Killeen:med chi-squared = 42.3, df = 5, p-value < 0.001
# → p < 0.05: variances are NOT equal
# → use non-parametric post-hoc (Dunn's test or pairwise Wilcoxon)
```

When to prefer Fligner-Killeen

If Shapiro-Wilk rejects normality in any group, prefer fligner.test() over bartlett.test(). If data is normal, Bartlett's test has more statistical power.

How to test for equality of variances with bartlett.test()

Bartlett's test is the classic parametric test for homogeneity of variances across groups. It is very sensitive to departures from normality — only use it when you have confirmed that the data within each group is approximately normal.

```
library(stats)  # loaded by default

# Bartlett's test (assumes normal distribution within groups)
bartlett.test(wvs6_rus$age ~ wvs6_rus$mar)
# Bartlett test of homogeneity of variances
# Bartlett's K-squared = 58.7, df = 5, p-value < 0.001
# → p < 0.05: variances are NOT equal
# → Only reliable here if data within each group is normal
```

Sensitivity warning

If your data is non-normal, Bartlett's test will often reject equal variances even when the variances are actually similar. In that case, use fligner.test() or leveneTest(

Made with Manus

How to run a Student's independent t-test (2 groups, equal variances)

Use Student's t-test when you compare the means of **two independent groups**, the data is approximately normal, and Levene's test confirms **equal variances** (p > 0.05). Pass `var.equal = TRUE` to use the pooled variance estimate.

```
# Are men and women different in age? (equal variances confirmed)
t.test(wvs6_rus$age ~ droplevels(as_factor(wvs6_rus$V240)), var.equal = TRUE)

# Two Sample t-test
# t = -2.14, df = 2483, p-value = 0.032
# alternative hypothesis: true difference in means is not equal to 0
# 95 percent confidence interval: -1.94  -0.08
# sample estimates:
# mean in group Female  mean in group Male
#                 44.2                44.8
# → p < 0.05: mean age differs significantly between men and women
```

Always check variances first

Run leveneTest() before choosing between Student's and Welch's t-test. If p < 0.05 in Levene's test, use Welch's t-test (var.equal = FALSE) instead.

How to run a Welch's t-test (2 groups, unequal variances)

Welch's t-test is the **safer default** for two independent groups when variances are unequal (Levene's p < 0.05). It adjusts the degrees of freedom using the Welch-Satterthwaite equation, making it more robust. Pass `var.equal = FALSE` (this is actually R's default).

Made with Manus

```
# Welch's t-test (unequal variances — R default)
t.test(wvs6_rus$age ~ droplevels(as_factor(wvs6_rus$V240)), var.equal = FALSE

# Welch Two Sample t-test
# t = -2.14, df = 2477.3, p-value = 0.032
# alternative hypothesis: true difference in means is not equal to 0
# 95 percent confidence interval: -1.94  -0.08
# sample estimates:
# mean in group Female  mean in group Male
#               44.2                44.8
# → Note: df is adjusted (2477.3 vs 2483 for Student's)
# → p < 0.05: significant difference in mean age
```

R default

In R, t.test() uses var.equal = FALSE by default, so Welch's t-test is what you get unless you explicitly pass var.equal = TRUE. When in doubt, Welch's is the safer choice.

How to run a paired t-test (2 related measurements)

Use the paired t-test when the **same subjects** are measured twice (before/after, or two related variables). It tests whether the mean of the **differences** is zero. Before running it, check that the differences are approximately normal with `shapiro.test()`.

```
# Are financial satisfaction and life satisfaction different in Russia?
# Step 1: compute the difference and check normality
wvs6_rus$satdif ← wvs6_rus$finsat - wvs6_rus$lifesat
shapiro.test(wvs6_rus$satdif)
# W = 0.972, p-value < 0.001 → differences are NOT normal
# → use Wilcoxon Signed Rank test instead (see next topic)

# Step 2 (if normal): run paired t-test
t.test(wvs6_rus$finsat, wvs6_rus$lifesat, paired = TRUE)
# Paired t-test
# t = -12.4, df = 2484, p-value < 0.001
# mean of differences: -0.48
# → Financial satisfaction is significantly lower than life satisfaction
```

Check normality of differences

The paired t-test assumes the pairwise differences are normally distribute
original variables. Compute the difference variable first and run shapiro.tes

Made with Manus

non-normal, use wilcox.test(..., paired = TRUE) instead.

How to run a Wilcoxon Signed Rank test (2 paired/related samples)

The Wilcoxon Signed Rank test is the **non-parametric alternative to the paired t-test**. Use it when the differences between two related measurements are not normally distributed, or when the outcome is ordinal. It ranks the absolute differences and tests whether the positive and negative ranks are balanced.

```
# Financial satisfaction vs life satisfaction in Russia
# (differences are non-normal → use Wilcoxon Signed Rank)
wilcox.test(wvs6_rus$finsat, wvs6_rus$lifesat, paired = TRUE)

# Wilcoxon signed rank test with continuity correction
# V = 456789, p-value < 0.001
# alternative hypothesis: true location shift is not equal to 0
# → p < 0.001: financial satisfaction and life satisfaction
#   differ significantly in Russia
```

Paired vs independent

Use paired = TRUE for two measurements on the same subjects (before/after, two related variables). Use paired = FALSE (the default) for two independent groups — that gives you the Mann-Whitney U test.

How to run Tukey HSD post-hoc test after ANOVA

After a significant ANOVA result, `TukeyHSD()` tests all pairwise group differences while controlling the family-wise error rate. Each row in the output shows the mean difference, its 95% confidence interval, and an adjusted p-value. Use this when variances are equal across groups.

```
# After a significant ANOVA, identify which pairs of groups differ
fit ← aov(wvs6_rus$age ~ wvs6_rus$mar)
summary(fit)  # confirm overall ANOVA is significant first

TukeyHSD(fit)
#                              diff    lwr    upr   p adj
# Married-Divorced            -2.14  -4.21  -0.07  0.038 *
# Single-Divorced             -9.82 -11.90  -7.74  <0.001 ***
# Widowed-Married             14.51  12.80  16.22  <0.001 ***
```

Made with Manus

```
# ...
# → Pairs with p adj < 0.05 have significantly different mean ages
# → 'diff' is the mean difference (Group B minus Group A)
# → Negative diff means Group A has a higher mean

plot(TukeyHSD(fit), las = 1)  # visual: CIs that cross 0 are not significant
```

Equal variances required

TukeyHSD() assumes equal variances. If Levene's test was significant (unequal variances), use games_howell_test() instead.

▸ How to run Games-Howell post-hoc test (unequal variances)

`games_howell_test()` from the **rstatix** package is the recommended post-hoc test after a Welch's ANOVA (unequal variances, unequal sample sizes). It does not assume equal variances and is robust to unbalanced group sizes.

```
library(rstatix)

# Post-hoc after Welch's ANOVA (unequal variances)
games_howell_test(data = wvs6_rus, age ~ mar, conf.level = 0.95)

# .y.  group1    group2        estimate conf.low conf.high  p.adj  p.adj.sig
# age  Divorced  Married         -2.14    -4.30     0.02   0.054  ns
# age  Divorced  Single          -9.82   -11.95    -7.69  <0.001  ****
# age  Divorced  Widowed         12.37    10.10    14.64  <0.001  ****
# ...
# → 'estimate' is the mean difference (group2 minus group1)
# → p.adj.signif: ns = not significant, * p<0.05, ** p<0.01, etc.
```

When to use Games-Howell

Use games_howell_test() when Levene's test is significant (p < 0.05), indicating unequal variances. For equal variances, TukeyHSD() is the standard choice.

▸ How to run Dunn's test post-hoc after Kruskal-Wallis

Made with Manus

`kwAllPairsDunnTest()` from the **PMCMRplus** package performs Dunn's test — the standard non-parametric post-hoc after a significant Kruskal-Wallis result. It compares all pairs of groups based on rank sums and applies a p-value correction for multiple testing.

```
library(PMCMRplus)

# Post-hoc after significant Kruskal-Wallis
kruskal.test(age ~ mar, data = wvs6_rus)
# chi-squared = 892, df = 5, p-value < 0.001 → at least one pair differs

# Dunn's test with Benjamini-Hochberg correction
kwAllPairsDunnTest(wvs6_rus$age ~ wvs6_rus$mar, p.adjust.method = "BH")

#                Divorced  Married  Separated  Single  Widowed
# Married         0.038
# Separated       0.612    0.201
# Single         <0.001   <0.001    0.003
# Widowed        <0.001   <0.001    <0.001    <0.001
# Living together 0.180   0.041    0.612     <0.001   <0.001
# → Each cell is the adjusted p-value for that pair
# → p < 0.05: the two groups have significantly different rank distributions
```

p-value correction

The p.adjust.method argument controls the correction for multiple comparisons. "BH" (Benjamini-Hochberg) controls the false discovery rate and is less conservative than Bonferroni. Use "bonferroni" for stricter control.

▸ How to run Dunnett's test (comparing all groups to a reference)

`DunnettTest()` from the **DescTools** package compares each treatment group to a single reference (control) group, rather than all possible pairs. This is more powerful than Tukey HSD when you only care about comparisons against one baseline group (e.g., a placebo or a reference category).

```
library(DescTools)

# Compare all marital status groups to 'Single' as the reference
DunnettTest(wvs6_rus$age ~ relevel(wvs6_rus$mar, ref = "Single"))
```

Made with Manus

```
# Dunnett's test for comparing several treatments with a control
```

```
#                    diff    lwr.ci   upr.ci    pval
# Married-Single    10.21     8.54    11.88  <0.001 ***
# Divorced-Single    0.39    -2.15     2.93   0.982
# Separated-Single   3.18    -1.02     7.38   0.201
# Widowed-Single    24.72    22.90    26.54  <0.001 ***
# ...
# → Only comparisons against 'Single' are shown
# → Pairs with pval < 0.05 differ significantly from the reference
```

Choosing the reference group

By default, DunnettTest() uses the first factor level as the reference. Use relevel() beforehand to set a different reference group (e.g., relevel(group, ref = 'Single')).

10 How to Run Linear Regression

Linear regression models the relationship between a continuous outcome variable and one or more predictors. The workflow moves from simple bivariate models through a combined model to a full model with controls, then compares standardised slopes to assess relative importance.

lm() summary(lm()) relevel() stargazer() lm.beta() adjusted R^2 F-statistic

How to set a reference category for a nominal variable with relevel()

When a nominal (categorical) variable enters a regression, R automatically creates dummy variables — one for each category except the **reference category**, whose effect is absorbed into the intercept. Use `relevel()` to choose which category serves as the reference. The choice should be theoretically motivated: pick the category you want to compare all others against.

```
# V187 - Religious self-perception: 1=Religious, 2=Not religious, 3=Atheist
# Convert to a factor with labels
wvs5_bul$rel_person ← droplevels(as_factor(wvs5_bul$V187))

# Set "Not a religious person" as the reference category
# This lets us test: do religious people / atheists differ fro
wvs5_bul$rel_person ← relevel(wvs5_bul$rel_person, ref = "Not
```

Made with Manus

```
# Verify: the reference category should appear first in levels()
levels(wvs5_bul$rel_person)
# [1] "Not a religious person" "A religious person"     "A convinced atheist
# ↑ "Not a religious person" is first → confirmed as reference

# Same approach for sex: set Male as reference
wvs5_bul$sex ← droplevels(as_factor(wvs5_bul$V235))
wvs5_bul$sex ← relevel(wvs5_bul$sex, ref = "Male")
levels(wvs5_bul$sex)
# [1] "Male"   "Female"
# ↑ "Male" is first → confirmed as reference
```

Why reference category choice matters

The intercept in the regression output represents the predicted value of the outcome for a respondent in the reference category (with all other predictors at zero). Each dummy coefficient then represents the *difference* from that baseline. Choosing a substantively meaningful reference (e.g., "Not a religious person") makes the coefficients directly interpretable for your hypothesis.

How to fit a bivariate linear regression with lm()

Use `lm(outcome ~ predictor, data = df)` to fit a simple regression. Wrap it in `summary()` to see the coefficients, standard errors, t-values, p-values, R^2, and the overall F-test. Run separate bivariate models first — one per predictor of interest — before combining them into a multivariate model.

```
# Model 1a — H1: Does church attendance predict institutional confidence?
m1a ← lm(ins ~ attend, data = wvs5_bul)
summary(m1a)

# Call:
# lm(formula = ins ~ attend, data = wvs5_bul)
#
# Residuals:                                        ← distribution of model e
#      Min       1Q   Median       3Q      Max
# -0.40099 -0.17768 -0.02107  0.15562  0.64563
#
# Coefficients:
#             Estimate Std. Error t value Pr(>|t|)
# (Intercept)  0.35437    0.01315  26.943   <2e-16 ***   ← pre
# attend       0.04662    0.02923   1.595    0.111        ← sl
```

Made with Manus

```
# ---
# Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#
# Residual standard error: 0.2388 on 972 degrees of freedom
#   (27 observations deleted due to missingness)   ← listwise deletion of NAs
#
# Multiple R-squared:  0.002611    ← raw R²: attendance explains 0.26% of var
# Adjusted R-squared:  0.001585    ← penalises for number of predictors (use
# F-statistic: 2.545 on 1 and 972 DF,  p-value: 0.111   ← overall model not s
#
# How to read this output:
# 1. Coefficients table: check Estimate (direction), Std. Error, and Pr(>|t|)
# 2. Adjusted R-squared: proportion of variance explained (penalised for mode
# 3. F-statistic + p-value: is the overall model significantly better than in
# 4. Degrees of freedom: n - k - 1 (n = observations, k = number of predictor

# Model 1b — H2: Does importance of God predict institutional confidence?
m1b ← lm(ins ~ god_imp, data = wvs5_bul)
summary(m1b)

# Model 1c — H3: Does religious self-perception predict institutional confide
# rel_person is nominal; reference = "Not a religious person"
m1c ← lm(ins ~ rel_person, data = wvs5_bul)
summary(m1c)
# Intercept = predicted ins for "Not a religious person" (reference group)
# Slope for "A religious person" = difference vs. reference group
# Slope for "A convinced atheist" = difference vs. reference group
```

Reading the output

Intercept: predicted outcome when the predictor = 0. **Slope**: change in outcome for a one-unit increase in the predictor. **R^2**: share of variance in the outcome explained by the model. **F-statistic**: tests whether the model as a whole is significantly better than a model with no predictors (intercept only).

How to build a combined model

After running separate bivariate models, combine all predictors of interest into one model to check whether their effects hold simultaneously. This reveals shared variance and potential multicollinearity between predictors. Compare Adjusted R^2 with the bivariate models to see whether combining them improves fit.

Made with Manus

```
# M2: combined model — all three religiosity measures together
```

```
m2 <- lm(ins ~ attend + god_imp + rel_person, data = wvs5_bul)
summary(m2)

# Call:
# lm(formula = ins ~ attend + god_imp + rel_person, data = wvs5_bul)
#
# Residuals:
#      Min       1Q   Median       3Q      Max
# -0.40295 -0.16448 -0.01513  0.15546  0.65641
#
# Coefficients:
#                                Estimate Std. Error t value Pr(>|t|)
# (Intercept)                    0.344471   0.019035  18.097   <2e-16 ***
# attend                        -0.005267   0.035528  -0.148    0.882    ← wa
# god_imp                        0.043526   0.034553   1.260    0.208    ← wa
# rel_personA religious person   0.016705   0.022462   0.744    0.457    ← wa
# rel_personA convinced atheist -0.040371   0.039661  -1.018    0.309
# ---
# Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#
# Residual standard error: 0.2402 on 850 degrees of freedom
#   (146 observations deleted due to missingness)   ← more NAs than bivariate
#
# Multiple R-squared:  0.00947    Adjusted R-squared:  0.004808
# F-statistic: 2.032 on 4 and 850 DF,  p-value: 0.08808
#
# Key interpretation:
# All three religiosity predictors lose significance when combined
# → They share variance: once you know one, the others add little new informa
# Adjusted R2 (0.005) is barely higher than any single bivariate model
# → Combining them does not meaningfully improve explanatory power
# F-statistic p = 0.088: the combined model is only marginally significant ov
#
# Next step: add socio-demographic controls (age, sex, income, education) as
# to test whether religiosity effects survive adjustment for confounders
```

▸ How to display a formatted regression table with stargazer()

> `stargazer()` prints publication-style regression tables that place multiple models side by side for easy comparison. Use `type = "text"` for console output during analysis. The table shows coefficients, standard errors (in parentheses), significance stars, and model fit statistics (R^2, N) for every model at once.

Made with Manus

```
library(stargazer)

# Compare the three bivariate models side by side
# Pass model objects as arguments; use type = "text" for console output
stargazer(m1a, m1b, m1c,
          type    = "text",
          dep.var.caption = "Institutional confidence",
          title   = "Models 1a–1c: Separate religiosity measures")

# Output:
# Models 1a–1c: Separate religiosity measures
# ==========================================================================
#                                          Institutional confidence
#                                 ------------------------------------------
#                                                      ins
#                                        (1)                  (2)
# --------------------------------------------------------------------------
# attend                                0.047
#                                      (0.029)
#
# god_imp                                                   0.064**
#                                                           (0.025)
#
# rel_personA religious person
#
#
# rel_personA convinced atheist
#
#
# Constant                            0.354***             0.341***
#                                      (0.013)              (0.015)
#
# --------------------------------------------------------------------------
# Observations                           974                  948
# R2                                    0.003                0.007
# Adjusted R2                           0.002                0.006
# Residual Std. Error            0.239 (df = 972)     0.240 (df = 946)      (
# F Statistic                   2.545 (df=1;972)  6.404** (df=1;946)  3.920*
# ==========================================================================
# Note:                                                        *p<0.1; *
#
# How to read this table:
# - Each column = one model; rows = predictors
# - Coefficient above, standard error below in parentheses
```

Made with Manus

```
# - Stars indicate significance level: *** p<0.01, ** p<0.05, * p<0.1
# - Compare Adjusted R2 across columns to see which model explains more varia
# - Compare F Statistic to check whether each model is significant overall
```

Tip — type argument

Use `type = "text"` for quick console inspection, `type = "html"` to embed in an R Markdown report, or `type = "latex"` for a LaTeX document. All produce the same table structure, only the output format differs.

How to compare standardised beta coefficients with lm.beta()

Raw regression slopes are only comparable when all predictors are on the same scale. **Standardised beta coefficients** (from `lm.beta()`) express each slope in standard deviation units, making it possible to directly compare the *relative importance* of predictors regardless of their original measurement scale. The predictor with the largest absolute standardised beta has the strongest association with the outcome.

```
library(lm.beta)

# Standardised beta coefficients for the full model
summary.lm.beta(lm.beta(m3))
# Output adds a 'Standardized' column to the usual summary() table
# Example:
#              Estimate Standardized Std. Error t value Pr(>|t|)
# attend        0.085    0.112        0.020      4.2    <0.001 ***
# god_imp       0.062    0.089        0.018      3.4    0.001  **
# rel_personReligious  0.031  0.055  0.015  2.1  0.038 *
# → attend has the largest standardised beta → strongest predictor

# Alternative: fit model with all continuous IVs already on [0, 1] scale
m3_rescaled ← lm(ins ~ attend + god_imp + rel_person + age01 + sex + income
                 data = wvs5_bul)
summary(m3_rescaled)
# Slopes now represent full-range effects (min → max of each predictor)
```

Alternative: rescale all continuous variables to [0, 1]

Another approach is to rescale every continuous predictor to [0, 1] before fitting the model (see Section 5). The resulting slopes then represent the *maximum possible effect* — the change in the outcome when the predictor moves from its minimum to its maximum. Both methods allow cross-predictor comparison; standardised ' conventional in academic reporting.

Made with Manus

11 How to Run Logistic Regression

Logistic regression models the probability of a binary outcome (0/1). Unlike linear regression, the dependent variable is the log-odds (logit) of the event occurring. Coefficients are interpreted as changes in log-odds, but are more intuitively read after exponentiation as odds ratios.

glm() family=binomial ifelse() exp() NagelkerkeR2() AIC() odds ratio log-odds

▸ How to create a binary (0/1) outcome variable with ifelse()

> Logistic regression requires a **binary numeric outcome** (0 = event did not occur, 1 = event occurred). Use `ifelse()` to recode an existing variable into 0/1, then verify the recoding with a cross-tabulation. Optionally attach labels with `labelled()` for readability.

```
# Voting in elections (V227): 1 = voted, 2 = didn't vote, 3–10 = other/NA
# Recode: 1 or 2 → 1 (voted), anything else → 0
wvs6_rus$vote ← ifelse(wvs6_rus$V227 < 3, 1, 0)

# Verify recoding with a cross-tabulation
table(wvs6_rus$V227, wvs6_rus$vote)
#        0    1
# 1      0  831   ← all "voted" correctly coded as 1
# 2      0  412   ← all "didn't vote" correctly coded as 1
# 3    214    0   ← "not eligible" correctly coded as 0
# ...

# Optionally add labels for display
library(haven)
wvs6_rus$vote ← labelled(wvs6_rus$vote, c("Not vote" = 0, "Vote" = 1))
table(as_factor(wvs6_rus$vote))
```

Check your recoding

Always cross-tabulate the original and new variable to confirm every original value maps to the correct binary code. Miscoded outcomes will silently produce wrong results.

Made with Manus

How to fit a binary logistic regression model with glm()

Use `glm()` with `family = "binomial"` to fit a logistic regression. The formula syntax is identical to `lm()`: `outcome ~ predictor1 + predictor2 + ...`. The model estimates the log-odds of the outcome being 1 for each combination of predictor values.

```
# Fit a logistic regression model
# DV: vote (0/1) — did the respondent vote?
# IVs: trust in government, trust in parties, trust in parliament, age, gende
binomial ← glm(vote ~ trgov + trparty + trparliam + age + gnd,
               family = "binomial",
               data = wvs6_rus)

summary(binomial)

# Call:
# glm(formula = vote ~ trgov + trparty + trparliam + age + gnd,
#     family = "binomial", data = wvs6_rus)
#
# Coefficients:
#              Estimate Std. Error z value Pr(>|z|)
# (Intercept) -0.34387   0.23456  -1.466   0.143
# trgov        0.18421   0.06234   2.955   0.003 **
# trparty      0.09812   0.07123   1.378   0.168
# trparliam    0.11034   0.06891   1.601   0.109
# age          0.01823   0.00412   4.425  <0.001 ***
# gndMale     -0.08234   0.09123  -0.903   0.367
#
# (Dispersion parameter for binomial family taken to be 1)
#
# Null deviance: 1423.2 on 1052 degrees of freedom
# Residual deviance: 1389.4 on 1047 degrees of freedom
# AIC: 1401.4
#
# → Significant predictors: trust in government (p=0.003), age (p<0.001)
# → z value replaces t value (used for large-sample inference)
# → AIC: lower is better when comparing models
```

What is the logit?

The logit is the natural logarithm of the odds: logit(p) = ln(p / (1 – p)). The regression coefficients represent how the log-odds change with a one-unit increase in each predictor. Because log-odds are hard to interpret directly, they are usually to obtain odds ratios.

Made with Manus

How to interpret logistic regression coefficients as odds ratios

The raw coefficients from `summary(glm())` are in **log-odds** units, which are not intuitive. Exponentiating them with `exp()` converts them to **odds ratios**. An odds ratio > 1 means the predictor increases the odds of the event; < 1 means it decreases them; = 1 means no effect.

```
# Exponentiate all coefficients to get odds ratios
exp(coef(binomial))

# (Intercept)   trgov  trparty  trparliam     age  gndMale
#    0.7091    1.2023   1.1031    1.1166   1.0184   0.9210
#
# Interpretation:
# → Intercept (0.71): for the baseline profile (all IVs = 0, Female),
#   the odds of voting are 0.71 — i.e., less likely to vote than not
# → trgov (1.20): each 1-unit increase in trust in government
#   multiplies the odds of voting by 1.20 (+20%)
# → age (1.018): each additional year of age multiplies odds by 1.018
#   (+1.8% per year — older respondents vote more)
# → gndMale (0.92): men have 8% lower odds of voting than women
#   (not significant, p = 0.367)

# Manual calculation for the intercept:
2.71828^(-0.34387)  # = 0.709 — same as exp(coef(binomial)[1])
```

Intercept interpretation

The exponentiated intercept (exp(b0)) gives the odds of the event occurring when all predictors are zero. If the reference group is 'Male' and all other predictors are 0, exp(b0) < 1 means the probability of voting is less than 50% for that baseline profile.

How to compute Nagelkerke pseudo-R^2 for model fit

Logistic regression has no direct equivalent of R^2. **Nagelkerke's pseudo-R^2** from the `fmsb` package is the most widely reported approximation. It ranges from 0 to 1, where higher values indicate better fit. Values around 0.2–0.4 are considered acceptable in social science research. It is used for reporting, not for model comparison (use AIC for that).

```
library(fmsb)
```

Made with Manus

```
# Nagelkerke pseudo-R² for the full model
NagelkerkeR2(binomial)
# $N2
# [1] 0.0623
# → The model explains approximately 6.2% of the variance in voting
#   (by the Nagelkerke criterion)

# For comparison: pseudo-R² for a reduced model
NagelkerkeR2(binomial2)
# $N2
# [1] 0.0598
# → Slightly lower fit — dropping trparliam reduced explanatory power slighti
```

Pseudo-R² is not R²

Unlike linear regression R², pseudo-R² cannot be interpreted as 'proportion of variance explained'. It is a relative measure of fit improvement over a null model. Always report it alongside AIC and the significance of individual predictors.

How to compare logistic models with AIC

AIC (Akaike Information Criterion) balances model fit against complexity. The model with the **lower AIC** is preferred. Unlike pseudo-R², AIC penalises for adding predictors that do not improve fit, making it the standard tool for model selection in logistic regression. Use `AIC()` to compare any number of models.

```
# Fit a reduced model (dropping trparliam)
binomial2 ← glm(vote ~ trgov + trparty + age + gnd,
                family = "binomial",
                data = wvs6_rus)

# Compare AIC of both models
AIC(binomial)   # full model:    1401.4
AIC(binomial2)  # reduced model: 1399.8

# → binomial2 has lower AIC → statistically better model
# → removing trparliam improved fit (it was not contributing)
# → proceed with binomial2 as the preferred model
```

Rule of thumb

A difference of 2 or more AIC units is generally considered meaningful. A
below 2 suggests the models are roughly equivalent in fit. Always prefer the

Made with Manus

model when AIC values are close.

12 How to Calculate Confidence Intervals

A confidence interval (CI) gives a range of plausible values for a population parameter. The width of the interval depends on the sample size, the variability in the data, and the chosen confidence level.

qnorm() CI for mean CI for proportion tapply() cbind() rownames()

▸ How to compute CIs for a mean across multiple sample sizes

Vectorise the calculation by passing a vector of sample sizes. The critical value z is obtained from qnorm(). The correction n − 1 is used because we have a sample standard deviation.

```
mean_val  ← 5500
sd_sample ← 3000
z         ← qnorm(0.975)   # critical value for 95% CI (alpha = 0.05)

# Vector of sample sizes to compare
ssize ← c(100, 500, 1000, 10000, 20000, 50000)

# Lower and upper bounds (using n-1 correction for sample SD)
ci.l ← mean_val - z * sd_sample / sqrt(ssize - 1)
ci.u ← mean_val + z * sd_sample / sqrt(ssize - 1)

# Collect results in a data frame
ci_95_3000 ← data.frame(Lower = ci.l, Mean = mean_val, Upper = ci.u)
# rownames() assigns a descriptive label to each row
rownames(ci_95_3000) ← c("ss100", "ss500", "ss1000", "ss10000", "ss20000", '
ci_95_3000

# Repeat for a lower confidence level (alpha = 0.10, z = 1.645)
z90 ← qnorm(0.95)   # 90% CI
ci.l90 ← mean_val - z90 * sd_sample / sqrt(ssize - 1)
ci.u90 ← mean_val + z90 * sd_sample / sqrt(ssize - 1)
ci_90_3000 ← data.frame(Lower = ci.l90, Mean = mean_val, Uppe
```

Made with Manus

```
rownames(ci_90_3000) ← c("ss100", "ss500", "ss1000", "ss10000", "ss20000", '

# cbind() joins the two result tables side by side for easy comparison
all_ci ← cbind(ci_95_3000, ci_90_3000)
all_ci
# → Narrower intervals in the 90% table confirm that
#   reducing the confidence level shrinks the CI width
```

▸ How to compute group-wise CIs using tapply()

> tapply() applies a function to subsets of a vector defined by a grouping factor. Use it to compute group means and standard deviations before constructing CIs for each group.

```
z ← qnorm(0.975)   # 95% CI

# Group-wise mean and SD
gt_mean ← tapply(wvs6_rusw$trust_gen_cnt,
                 droplevels(as_factor(wvs6_rusw$V2)), mean, na.rm = TRUE)
gt_SD   ← tapply(wvs6_rusw$trust_gen_cnt,
                 droplevels(as_factor(wvs6_rusw$V2)), sd,   na.rm = TRUE)

# Valid sample sizes per group
gtval ← table(complete.cases(wvs6_rusw$trust_gen_cnt),
              droplevels(as_factor(wvs6_rusw$V2)))

# CI for Russia (index 1)
ci.l ← gt_mean[1] - z * gt_SD[1] / sqrt(gtval[2, 1] - 1)
ci.u ← gt_mean[1] + z * gt_SD[1] / sqrt(gtval[2, 1] - 1)
round(c(ci.l, ci.u), 3)
```

▸ How to compute a CI for a proportion

> For binary outcomes (e.g., share of unhappy people), the standard error is sqrt(p * (1 - p) / (n - 1)). This is the Wald interval — straightforward and widely used.

```
# Share of unhappy people per country (column proportions)
hap_share ← round(prop.table(
  table(wvs6_rusw$happy_rec,
        droplevels(as_factor(wvs6_rusw$V2))), 2), 3)
```

Made with Manus

```
p  ← hap_share[1, 1]   # proportion "not happy" in Russia
n  ← hap_val[2, 1]     # valid sample size for Russia
z  ← qnorm(0.975)

ci.l ← p - z * sqrt(p * (1 - p) / (n - 1))
ci.u ← p + z * sqrt(p * (1 - p) / (n - 1))
round(c(ci.l, ci.u), 3)
# With 95% probability, 22.3% to 25.7% of Russians are not happy
```

Key insight

CIs narrow as sample size increases, as population variability decreases, or as the confidence level decreases (i.e., higher alpha). Comparing non-overlapping CIs between groups provides informal evidence of a statistically significant difference.

R Data Wrangling & Statistics — Quick Reference Guide | Based on course home assignments (HA1–HA9, 2025)

Made with Manus